

SB3 代码导读：策略网络如何按环境自动搭建

Gymnasium + Stable-Baselines3 源码走读

深度强化学习课程 · 配套讲义

摘要

这份讲义带你读 **Stable-Baselines3 (SB3)** 的策略网络是怎么搭的：从你给的环境（动作空间 + 观测空间）出发，SB3 如何自动选出特征提取器、概率分布和网络头，把它们拼成一个可训练的策略。我们把代码结构和前面课上推过的理论（ $\nabla_{\theta} \log \pi$ 、softmax/高斯头、tanh-squash、on/off-policy、rollout/replay buffer）一一对应起来。代码片段为**简化示意**（抓逻辑，非逐字源码），每节给出真实文件位置，便于你打开仓库对照阅读。版本基准：SB3 2.x（已迁移到 Gymnasium），PyTorch 后端。

1 先建立心智模型：SB3 的“策略”是什么

一句话

在 SB3 里，“policy”不是 RL 术语里那个 π ，而是一整个**网络包**：特征提取器 + 策略/价值网络（+ 目标网络）+ 优化器，全打包在一个 policy 对象里。真正输出动作的只是其中一部分。

不管什么算法，前向数据流都是同一个三段式：

obs → **预处理**（图像归一化 / 离散 obs 转 one-hot）→ **特征提取器**（按观测空间选）
→ **net_arch 全连接干**（latent_pi, latent_vf）→ **分布头**（按动作空间选）→
action

两个“按空间分派”的点是全部魔法所在：

- **观测空间** ⇒ 决定特征提取器：向量 obs 用 `FlattenExtractor`，图像用 `NatureCNN`，字典用 `CombinedExtractor`。
- **动作空间** ⇒ 决定分布 + 网络头：离散用 `softmax/Categorical`，连续用高斯，等等。

源码位置

`MlpPolicy` / `CnnPolicy` / `MultiInputPolicy` 只是“按观测空间预选好特征提取器”的别名，三者最终都走同一套策略类。打印 `print(model.policy)` 能看到完整结构（见 §8）。

2 Gymnasium 这一侧：环境、空间、交互循环

后面几节讲 SB3 怎么消费动作/观测空间。但这些空间和交互数据全部来自 **Gymnasium** 环境 (Farama 接手 OpenAI Gym 后的继任者)。SB3 ≥ 2.0 用的是 `gymnasium` (`import gymnasium as gym`)，不是已弃用的老 `gym`。先看环境这侧给你什么。

2.1 两个方法走天下：reset 与 step

```
import gymnasium as gym

env = gym.make("CartPole-v1")
obs, info = env.reset(seed=0)          # -> (observation, info dict)

for _ in range(1000):
    action = env.action_space.sample()  # here: a random policy
    obs, reward, terminated, truncated, info = env.step(action)  # 5-tuple
    if terminated or truncated:         # episode finished -> reset
        obs, info = env.reset()
env.close()
```

- `reset()` 返回 (obs, info) 二元组 (老 gym 只返回 obs)。
- `step(action)` 返回 **5 元组** (obs, reward, terminated, truncated, info)。
- 还暴露 `env.observation_space`、`env.action_space`、`env.render()`、`env.close()`。

五元组逐个存什么：

#	名字	类型	存什么
1	observation	space 元素	下一个状态 s' (执行 action 后的新观测)
2	reward	float	即时奖励 r_t ，即 $(s, a) \rightarrow s'$ 拿到的回报
3	terminated	bool	到达 MDP 真终止态 (成功/失败/掉坑)
4	truncated	bool	因外部原因截断 (时间上限、人为中止), MDP 没真结束
5	info	dict	辅助信息：日志指标、调试量、隐藏变量；向量环境 <code>autoreset</code> 时真末状态放这 (<code>final_obs</code>)

(完全可观时 `observation = s'`；部分可观时它只是 s' 的一个视图，即 POMDP。)

对应你熟悉的 transition ($s, a, r, s', done$)

s 不在返回里——它是上一步的 obs (或 reset 的返回)，是本次 step 的输入； a 是你传进去的 action； r = reward； s' = observation； $done$ = terminated or truncated。所以存进 buffer 的一条样本通常是 (s, a, reward, observation, terminated)——自举掩码用 **terminated 而非 done**，truncated 只用来决定要不要 reset (理由见下框)。

terminated vs truncated: 为什么 Gym 要拆开 (直接关系自举)

v0.26 起 Gymnasium 把老的 done 拆成两个布尔:

- **terminated**: 到达 MDP 真正的终止态 (成功/失败/掉坑), 此后没有未来回报。
- **truncated**: 因外部原因被截断 (到达时间上限 TimeLimit、人为中止), MDP 本身没结束。

为什么关键? 自举目标

$$y = r + \gamma(1 - \text{terminated})V(s')$$

只能用 **terminated** 把 $V(s')$ 清零; 若是 **truncated** (时间到), s' 其实还有未来价值, 不该清零, 否则 bootstrapping 目标算错、loss 有偏。这正是前文 DQN/TD 自举里“是否乘 $(1 - \text{done})$ ”那个 done 的正确含义——它应当是 **terminated**, 不是 **terminated or truncated**。

2.2 空间 spaces: SB3 分派的全部依据

`gymnasium.spaces` 定义观测/动作的类型。后面“动作侧”“观测侧”两节里的 `isinstance` 判的就是这些类:

```
from gymnasium import spaces
import numpy as np

spaces.Box(low=-1.0, high=1.0, shape=(3,), dtype=np.float32)  # continuous vector
spaces.Discrete(4)  # one of {0,1,2,3}
spaces.MultiDiscrete([3, 5])  # one of {0..2} AND one of {0..4}
spaces.MultiBinary(6)  # 6 independent bits
spaces.Dict({"img": spaces.Box(0, 255, (3, 64, 64), np.uint8),
            "vec": spaces.Box(-1, 1, (10,))})  # composite obs

space = spaces.Box(-1, 1, (3,))
a = space.sample()  # draw a valid element
a in space  # membership / validity check
```

速记映射 (空间 $\rightarrow$ SB3 自动选什么, 详见后两节): `Box` $\rightarrow$ 高斯/压缩高斯头; 图像 `Box` $\rightarrow$ NatureCNN; `Discrete` $\rightarrow$ Categorical(softmax); `MultiDiscrete` $\rightarrow$ 多 softmax; `MultiBinary` $\rightarrow$ 伯努利; `Dict` $\rightarrow$ CombinedExtractor。

2.3 写一个自定义环境

继承 `gym.Env`, 在 `__init__` 里定义两个 space, 实现 `reset` 与 `step`:

```
import gymnasium as gym
from gymnasium import spaces
import numpy as np

class GoLeftEnv(gym.Env):
    """Toy: a 1-D line of length N; goal is to reach the left end (state 0)."""
    metadata = {"render_modes": []}
```

```

def __init__(self, size=10):
    super().__init__()
    self.size = size
    self.observation_space = spaces.Box(0, size, shape=(1,), dtype=np.float32)
    self.action_space = spaces.Discrete(2)          # 0 = left, 1 = right

def reset(self, *, seed=None, options=None):
    super().reset(seed=seed)                        # seeds self.np_random
    self.pos = self.size
    return np.array([self.pos], dtype=np.float32), {}

def step(self, action):
    self.pos = np.clip(self.pos + (-1 if action == 0 else 1), 0, self.size)
    terminated = bool(self.pos == 0)                # reached the goal
    truncated = False                               # TimeLimit wrapper may set this
    reward = 1.0 if terminated else -0.01
    return np.array([self.pos], dtype=np.float32), reward, terminated, truncated, {}

```

先用 SB3 的检查器验收

写完务必跑 `stable_baselines3.common.env_checker.check_env`，它校验 space 类型、reset/step 返回格式、dtype、5 元组是否合规——比训练崩了再 debug 省事得多。

```

from stable_baselines3.common.env_checker import check_env
from stable_baselines3 import PPO

env = GoLeftEnv()
check_env(env)                                # raises if the env is non-conformant
model = PPO("MlpPolicy", env).learn(10_000)

# register to create by id (max_episode_steps -> auto TimeLimit, i.e. sets truncated)
gym.register(id="GoLeft-v0", entry_point=GoLeftEnv, max_episode_steps=100)
env = gym.make("GoLeft-v0")

```

2.4 向量化与 SB3 的接管

RL 要海量样本，单环境太慢，所以并行跑多个副本、一次 step 一批：

```

# Gymnasium native:
envs = gym.make_vec("CartPole-v1", num_envs=8, vectorization_mode="sync") # or "async"
obs, info = envs.reset(seed=0)      # obs shape: (8, 4)
obs, rewards, terminated, truncated, infos = envs.step(envs.action_space.sample())

```

autoreset：批里某个子环境结束了怎么办

向量环境会自动重置结束的子环境（默认 next-step 模式）。结束那一步真正的末状态放在 `infos["final_obs"]` 里——别拿被重置后的首状态当 s' 去算自举目标，要用 `final_obs`

(与上面 `terminated` 的道理一脉相承)。

SB3 自带一套 `VecEnv` (为兼容历史, `step` 返回 `(obs, rewards, dones, infos)`, `done = terminated or truncated`, 末状态放 `info["terminal_observation"]`):

```
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.vec_env import SubprocVecEnv, VecNormalize

venv = make_vec_env("CartPole-v1", n_envs=8)  # DummyVecEnv by default
venv = make_vec_env("CartPole-v1", n_envs=8, vec_env_cls=SubprocVecEnv)  # true multiprocessing
venv = VecNormalize(venv, norm_obs=True, norm_reward=True)  # running normalization
```

对应前文理论

`DummyVecEnv` (同进程串行) vs `SubprocVecEnv` (多进程真并行) 对应前文“向量化环境”那节: CPU 仿真是瓶颈时用 `SubprocVecEnv`。给 SB3 传字符串或单个 `env` 时, 它会自动套上 `Monitor` + `DummyVecEnv`, 所以你拿到的 `obs` 永远带 `batch` 维 (`n_envs`, `*obs_shape`)。

2.5 Wrappers: 不改环境就改输入输出

`gymnasium.Wrapper` 及三个子类按需改造: `ObservationWrapper` / `RewardWrapper` / `ActionWrapper` 分别改观测/奖励/动作。常用现成件: `TimeLimit` (加 `truncated`)、`RecordEpisodeStatistics`、`FlattenObservation`、`FrameStack`、`AtariPreprocessing` (灰度/缩放/跳帧)。SB3 侧还有 `VecNormalize`、`VecFrameStack`、`VecTransposeImage` (`HWC`→`CHW`)。

2.6 端到端: 一句话把两侧连起来

```
import gymnasium as gym
from stable_baselines3 import PPO

# Gym side: defines obs/action spaces, reward, terminated/truncated
env = gym.make("LunarLander-v3")  # Box(8) obs, Discrete(4) action
# SB3 side: reads those spaces -> Flatten extractor + Categorical head + value head
model = PPO("MlpPolicy", env, verbose=1)
model.learn(200_000)
```

两侧分工

Gym/Gymnasium 负责环境: 定义 `observation/action space`、在 `step` 里给出 `reward` 与 `terminated/truncated`。**SB3** 负责智能体: 读这两个 `space` 自动搭网络 (见后两节), 跑交互循环、填 `buffer`、更新参数。你上新问题时, 绝大多数工作量在把问题表达成一个合规的 `gym.Env` (设计 `state/action/reward`), 网络那侧 SB3 基本自动。

3 动作侧：分布怎么按动作空间分派

这是和前面理论最直接挂钩的一段。整个分派集中在一个工厂函数里：

源码位置

stable_baselines3/common/distributions.py → make_proba_distribution()

```
# simplified: action_space -> probability distribution
from gymnasium import spaces

def make_proba_distribution(action_space, use_sde=False, dist_kwargs=None):
    dist_kwargs = dist_kwargs or {}
    if isinstance(action_space, spaces.Box):          # continuous
        cls = StateDependentNoiseDistribution if use_sde else DiagGaussianDistribution
        return cls(get_action_dim(action_space), **dist_kwargs)
    elif isinstance(action_space, spaces.Discrete):    # one categorical choice
        return CategoricalDistribution(int(action_space.n), **dist_kwargs)
    elif isinstance(action_space, spaces.MultiDiscrete): # several independent categoricals
        return MultiCategoricalDistribution(list(action_space.nvec), **dist_kwargs)
    elif isinstance(action_space, spaces.MultiBinary): # several independent bits
        return BernoulliDistribution(int(action_space.n), **dist_kwargs)
    else:
        raise NotImplementedError(f"unsupported action space {action_space}")
```

动作空间 (Gymnasium)	分布类	对应概率模型
Discrete(n)	CategoricalDistribution	softmax over n
MultiDiscrete	MultiCategoricalDistribution	多个独立 softmax
MultiBinary(n)	BernoulliDistribution	n 个独立伯努利
Box (连续)	DiagGaussianDistribution	对角高斯
Box + use_sde=True	StateDependentNoiseDistribution	gSDE (状态相关噪声)
Box (SAC/TD3 内部)	SquashedDiagGaussian	tanh 压缩的高斯

3.1 离散：Categorical 头 = 一个线性层 + softmax

每个分布类都有个 `proba_distribution_net()`，负责造网络头（把 latent 映射成分布参数），另有 `sample/mode/log_prob/entropy` 负责采样与梯度：

```
class CategoricalDistribution(Distribution):
    def __init__(self, action_dim):
        self.action_dim = action_dim
    def proba_distribution_net(self, latent_dim):
        # the "softmax head": latent -> one logit per action
        return nn.Linear(latent_dim, self.action_dim)
    def proba_distribution(self, action_logits):
        self.distribution = th.distributions.Categorical(logits=action_logits)
        return self
```

```

def log_prob(self, actions): return self.distribution.log_prob(actions)
def entropy(self):          return self.distribution.entropy()
def sample(self):           return self.distribution.sample()
def mode(self):             return th.argmax(self.distribution.probs, dim=1)

```

对应前文理论

注意 `nn.Linear` 出的是 *logits*, softmax 和 $\log \pi$ 、熵、采样全交给 PyTorch 的 `th.distributions.Categorical` —— $\nabla_{\theta} \log \pi(a|s)$ 由 autograd 自动算, 正是前文那个 score。课上推的 $\partial \log \pi(a) / \partial z_b = \mathbb{I}[a=b] - \pi(b)$ 就藏在 `Categorical.log_prob` 的反向里。

3.2 连续: 高斯头 = 线性均值 + 一个独立的 log_std 参数

```

class DiagGaussianDistribution(Distribution):
    def __init__(self, action_dim):
        self.action_dim = action_dim

    def proba_distribution_net(self, latent_dim, log_std_init=0.0):
        mean_actions = nn.Linear(latent_dim, self.action_dim)  # state-DEPENDENT
        # mean
        log_std = nn.Parameter(th.ones(self.action_dim) * log_std_init)  # state-INDEPENDENT
        # std!
        return mean_actions, log_std

    def proba_distribution(self, mean_actions, log_std):
        self.distribution = th.distributions.Normal(mean_actions, log_std.exp())
        return self

    def log_prob(self, actions):
        # independent dims -> sum log-probs over action dimensions
        return sum_independent_dims(self.distribution.log_prob(actions))

    def entropy(self):
        return sum_independent_dims(self.distribution.entropy())

    def sample(self):
        # reparameterized sample: mu + sigma * eps
        return self.distribution.rsample()

```

对应前文理论

这正是前文说的“连续策略 = MLP 出 $\mu_{\theta}(s)$, 配一个标准差 σ ”, 而且 SB3 默认用状态无关的 σ : `log_std` 是个 `nn.Parameter` (整段训练共享一个向量), 不由网络按状态输出。 $\nabla_{\mu} \log \pi = (a - \mu) / \sigma^2$ 同样由 autograd 给出。

3.3 SAC 的 tanh 压缩高斯: 多一个换元修正

有界连续动作(如力矩 $\in [-1, 1]$)不能用裸高斯。SAC 用 `SquashedDiagGaussianDistribution`: 先采高斯 u , 再 $a = \tanh(u)$, log-prob 要加 $\tanh$ 的 Jacobian 修正:

```

class SquashedDiagGaussianDistribution(DiagGaussianDistribution):
    def sample(self):
        self.gaussian_actions = super().sample()  # u ~ N(mu, sigma)

```

```

    return th.tanh(self.gaussian_actions)          # a = tanh(u) in (-1, 1)
def log_prob(self, actions, gaussian_actions=None):
    if gaussian_actions is None:                  # invert tanh if needed
        gaussian_actions = TanhBijector.inverse(actions)
    log_prob = super().log_prob(gaussian_actions)
    # change-of-variables correction: -sum log(1 - tanh(u)^2)
    log_prob -= th.sum(th.log(1 - actions ** 2 + 1e-6), dim=1)
    return log_prob

```

对应前文理论

对应前文“连续动作有界 $\rightarrow$ tanh-squash, log-prob 要加 Jacobian 修正项”。修正项 $-\sum_i \log(1 - a_i^2)$ 就是 $|\det \partial \tanh / \partial u|^{-1}$ 取对数。

4 观测侧：特征提取器怎么按观测空间选

源码位置

stable_baselines3/common/torch_layers.py (FlattenExtractor, NatureCNN, CombinedExtractor, MlpExtractor) ; 判别图像用 common/preprocessing.py $\rightarrow$ is_image_space

```

class FlattenExtractor(BaseFeaturesExtractor):      # vector / low-dim obs
    def __init__(self, obs_space):
        super().__init__(obs_space, get_flattened_obs_dim(obs_space))
        self.flatten = nn.Flatten()
    def forward(self, obs):
        return self.flatten(obs)                  # the "extractor" is basically a no-op

class NatureCNN(BaseFeaturesExtractor):            # image obs (Mnih et al. 2015)
    def __init__(self, obs_space, features_dim=512):
        super().__init__(obs_space, features_dim)
        n_c = obs_space.shape[0]                  # channels-first (C, H, W)
        self.cnn = nn.Sequential(
            nn.Conv2d(n_c, 32, 8, stride=4), nn.ReLU(),
            nn.Conv2d(32, 64, 4, stride=2), nn.ReLU(),
            nn.Conv2d(64, 64, 3, stride=1), nn.ReLU(), nn.Flatten())
        # one dummy forward to get flattened size, then a Linear to features_dim
        self.linear = nn.Sequential(nn.Linear(self._n_flatten(obs_space), features_dim), nn.ReLU())
    def forward(self, obs):
        return self.linear(self.cnn(obs))

```

- 1D 向量 obs (如 CartPole 的 Box(4,))：提取器就是 Flatten, 真正干活的是后面的 net_arch 全连接。

- **图像 obs** (如 Atari 的 Box(4,84,84)): 用 NatureCNN。on-policy (A2C/PPO) 默认 **actor/critic 共享 CNN** 省算力; off-policy (SAC/TD3) actor 和 critic **各用一套提取器** (实测效果更好)。
- **字典 obs** (Dict, 多模态/HER): 用 CombinedExtractor——图像 key 走 CNN, 向量 key 直接 flatten, 最后拼成一个长向量。

预处理悄悄做的事

喂进提取器前 SB3 会预处理: 图像归一化到 $[0, 1]$; **离散观测自动转 one-hot**; 向量 obs 仅 flatten。所以你自定义网络拿到的已经是干净的浮点张量。

5 把两侧拼起来: on-policy 的 ActorCriticPolicy

A2C / PPO / TRPO 都用 ActorCriticPolicy。它把“特征提取器 + 共享干 + 分布头 + 价值头”组装起来:

源码位置

stable_baselines3/common/policies.py → ActorCriticPolicy._build / forward / _get_action_dist_from_latent / evaluate_actions

```
class ActorCriticPolicy(BasePolicy):          # A2C / PPO / TRPO (simplified)
    def _build(self, lr_schedule):
        self.features_extractor = self.make_features_extractor()    # Flatten / CNN /
        Combined
        self.mlp_extractor = MlpExtractor(self.features_dim, self.net_arch, self.activation_fn)
        latent_pi = self.mlp_extractor.latent_dim_pi
        # ---- action head: chosen by the distribution type ----
        if isinstance(self.action_dist, DiagGaussianDistribution):
            self.action_net, self.log_std = self.action_dist.proba_distribution_net(
                latent_pi, log_std_init=self.log_std_init)
        elif isinstance(self.action_dist, CategoricalDistribution):
            self.action_net = self.action_dist.proba_distribution_net(latent_pi)
        # (MultiCategorical / Bernoulli / gSDE handled similarly)
        # ---- value head ----
        self.value_net = nn.Linear(self.mlp_extractor.latent_dim_vf, 1)

    def forward(self, obs, deterministic=False):
        features = self.extract_features(obs)
        latent_pi, latent_vf = self.mlp_extractor(features)    # shared / split trunk
        dist = self._get_action_dist_from_latent(latent_pi)
        actions = dist.get_actions(deterministic=deterministic)
        log_prob = dist.log_prob(actions)
        values = self.value_net(latent_vf)
        return actions, values, log_prob    # <-- exactly what goes into the rollout
        buffer
```

```
def _get_action_dist_from_latent(self, latent_pi):
    out = self.action_net(latent_pi)  # logits OR mean
    if isinstance(self.action_dist, DiagGaussianDistribution):
        return self.action_dist.proba_distribution(out, self.log_std)
    elif isinstance(self.action_dist, CategoricalDistribution):
        return self.action_dist.proba_distribution(action_logits=out)
```

对应前文理论

forward() 返回的 (actions, values, log_prob) 正是前文 rollout buffer 里存的三样东西。PPO 更新时调 evaluate_actions(obs, actions) 对 minibatch 重算 log_prob / entropy / values, 再配旧 log_prob 算重要性比 $\pi_\theta / \pi_{\text{old}}$ ——这就是前文“近端 off-policy、跑 K 个 epoch”的代码落点。net_arch=dict(pi=[...], vf=[...]) 控制 actor/critic 各自（或共享）的干。

6 off-policy 三家：没有分布 / 有压缩高斯 / 确定性

6.1 DQN：根本没有分布，输出每个动作的 Q

源码位置

stable_baselines3/dqn/policies.py → QNetwork

```
class QNetwork(BasePolicy):  # DQN: discrete actions only
    def __init__(self, obs_space, action_space, net_arch, ...):
        self.features_extractor = ...
        # MLP: features -> Q-value for EACH discrete action
        self.q_net = nn.Sequential(*create_mlp(self.features_dim, action_space.n, net_arch))
    def forward(self, obs):
        return self.q_net(self.extract_features(obs))  # shape (batch, n_actions)
    def _predict(self, obs, deterministic=True):
        return self.forward(obs).argmax(dim=1)  # greedy; epsilon handled in DQN.predict()
```

对应前文理论

对应前文 DQN 那节：输出 $Q(s, \cdot)$ 的一整排值、argmax 选动作，只支持 Discrete； ϵ -greedy 在外层 predict 里加；target network、replay buffer 在 DQN 算法类里维护，不在策略类里。

6.2 SAC：状态相关 std 的压缩高斯 + 双 Q

源码位置

stable_baselines3/sac/policies.py → Actor (演员)、ContinuousCritic (评论家)

```
class Actor(BasePolicy):  # SAC actor: continuous only
    def __init__(self, ...):
        self.latent_pi = nn.Sequential(*create_mlp(self.features_dim, -1, net_arch))  # trunk
```

```

self.mu = nn.Linear(last_dim, action_dim)
self.log_std = nn.Linear(last_dim, action_dim) # state-DEPENDENT std (cf. on-policy
Parameter)
self.action_dist = SquashedDiagGaussianDistribution(action_dim)
def forward(self, obs, deterministic=False):
    latent = self.latent_pi(self.extract_features(obs))
    mean = self.mu(latent)
    log_std = self.log_std(latent).clamp(LOG_STD_MIN, LOG_STD_MAX) # numerical safety
    return self.action_dist.actions_from_params(mean, log_std, deterministic)

```

两点和 on-policy 不同: (i) log_std 是网络输出、状态相关的 (PPO 是全局 Parameter); (ii) 评论家 ContinuousCritic 是 n 个并列 Q 网络 (默认 2 个, twin-Q 抗高估), 且 actor/critic 用独立特征提取器。

6.3 TD3/DDPG: 确定性演员 + tanh

演员直接输出动作 $a = \tanh(\text{MLP}(s))$ (无分布、确定性), 评论家是 Q 网络。和 SAC 共用 ContinuousCritic。

7 不同环境 → 自动选了什么: 速查表

环境 (例)	观测空间	动作空间	特征提取器/分布	常用算法
CartPole	Box(4,)	Discrete(2)	Flatten / Categorical	PPO,A2C,DQN
Pendulum	Box(3,)	Box(1,)	Flatten / 高斯 (或压缩)	PPO,SAC,TD3
LunarLander-cont.	Box(8,)	Box(2,)	Flatten / 高斯	PPO,SAC
Atari (Breakout)	Box(4,84,84)	Discrete	NatureCNN / Categorical	PPO,DQN
MultiDiscrete 任务	视情况	MultiDiscrete	*/多 softmax	PPO,A2C
机器人 + 目标 (HER)	Dict	Box	CombinedExtractor / 压缩高斯	SAC,TD3

默认网络大小 (1D obs)

PPO/A2C/DQN: 两层 $\times 64$; SAC: 两层 $\times 256$; TD3/DDPG: [400, 300] (沿用原论文)。图像一律先过 NatureCNN。

联系你的运筹方向

路由 / 调度 / DVRP 这类组合、可变、结构化动作空间, SB3 原生分布 (flat softmax / 高斯) 搞不定。实务路线: (a) MaskablePPO (在 sb3-contrib 里) ——对不可行动作做 logit 屏蔽, 仍是 Categorical 但带 mask; (b) 超出 mask 的 (pointer/attention 序贯解码) 就得自定义策略类, 或转用更灵活的框架。 $\nabla \log \pi$ 可微可采样这个内核不变, 变的只是“头”的结构。

8 怎么真正打开仓库读: 路线与技巧

推荐阅读顺序 (自底向上, 每个文件只看上面点到的类/函数):

1. `common/distributions.py` ——先看 `make_proba_distribution`, 再看 `Categorical/DiagGaussian` 的 `proba_distribution_net`。这是“动作空间 → 头”的核心。
2. `common/torch_layers.py` ——`FlattenExtractor` / `NatureCNN` / `MlpExtractor` / `create_mlp`。这是“观测空间 → 特征 + 全连接干”。
3. `common/policies.py` ——`BasePolicy` 与 `ActorCriticPolicy`, 看 `_build` 怎么把上面两块拼起来、`forward` 怎么产出 `(action, value, log_prob)`。
4. 再按需读 `dqn/policies.py`、`sac/policies.py`、`td3/policies.py` 看 off-policy 的差异。

最快的“看清结构”技巧——直接打印策略对象，把上面所有抽象变成具体层：

```
import gymnasium as gym
from stable_baselines3 import PPO

m = PPO("MlpPolicy", "CartPole-v1")
print(m.policy)           # whole bundle: features_extractor + mlp_extractor + action_net
                          + value_net
print(m.policy.action_dist) # CategoricalDistribution(...)
print(m.policy.action_net)  # Linear(in_features=64, out_features=2)  <- the softmax head

m2 = PPO("MlpPolicy", "Pendulum-v1")
print(m2.policy.action_dist) # DiagGaussianDistribution(...)
print(m2.policy.action_net)  # Linear(in_features=64, out_features=1)  <- the mean head
print(m2.policy.log_std)     # Parameter of shape (1,)  <- state-independent std
```

在线源码

逐字源码可在 readthedocs 的 “[source]” 链接或 GitHub `DLR-RM/stable-baselines3` 看。建议对照本讲义的简化版读真实实现，注意它多出来的初始化、类型注解、设备/数据类型处理等工程细节——逻辑骨架与这里一致。

9 一页纸小结

把整条链记住

环境给你两个空间。**观测空间**选特征提取器（向量 Flatten / 图像 CNN / 字典 Combined）；**动作空间**经 `make_proba_distribution` 选分布与头（离散 softmax / 连续高斯 / SAC 压缩高斯 / DQN 无分布直接 Q）。中间的 `net_arch` 全连接干把特征变成 `latent_pi`, `latent_vf`。on-policy 的 `ActorCriticPolicy.forward` 吐出 `(action, value, log_prob)` 灌进 rollout buffer；off-policy 各算法在自己的策略类里搭 actor/critic (+target) 配 replay buffer。前文所有理论（score、高斯/softmax 头、tanh-squash、on/off-policy、两类 buffer）在这套代码里都能一一对上号。